AI Tutor - Team Handbook

Everything you need, in one document. SOAIDEATHON-S28 - Track: Software - Category: Smart Education - 6 people - 36 hours.

Repository: https://github.com/SushreeSudiptaJena/AI-Tutor

---

1. What we are building

An AI tutor that teaches **only** from course material an admin has uploaded and approved. It shows the exact book and page behind every explanation, and when the approved material does not cover something, it says so instead of guessing.

The official problem statement:

> *Develop an AI tutor that uses approved course content, identifies prerequisite gaps, generates adaptive explanations and practice, and cites the source material used. It must detect when it lacks evidence, avoid completing graded work on behalf of students, support multilingual and accessible interaction, and give teachers dashboards focused on misconceptions rather than surveillance.*

Why this is not just a chatbot with a PDF attached

Anyone can put a textbook into a chatbot. Four things make this different, and they are exactly what the judges are scoring:

1. **Alignment score.** Every explanation shows a percentage saying how directly it maps to approved syllabus material. A number the student can see and trust.
2. **It refuses when it has no evidence.** If the curriculum does not cover the question, it says "I don't have approved material on this" instead of inventing an answer. That refusal is logged for the teacher rather than failing silently.
3. **It will not do graded work.** Asked to solve an assignment question, it declines and gives scaffolding instead of the answer.
4. **Teacher dashboards show misconceptions, not surveillance.** No time-on-task, no single score, no student names. What a teacher sees is "14 students think constant velocity implies a net force" - something they can actually teach against.

Three dashboards

- **Management (admin)** - uploads and structures the approved curriculum. Upload by a verified admin counts as approval for this build.
- **Student** - course scope, gap diagnosis, lessons, practice, misconception checks.
- **Teacher** - misconception heatmap, gap map, uncertainty flags, reteach approval.

---

2. The golden path

This is the one flow that must work **live**, end to end, on demo day. Everything else can be realistic pre-loaded data.

> *A student takes the prerequisite diagnostic -> sees a gap with its alignment score -> gets a lesson with a real*

> *book and page citation -> answers a practice question wrong -> the AI names the specific misconception behind that wrong answer -> the student confirms it -> the teacher's misconception heatmap updates live on a second laptop.*

Every decision in this project is subordinate to protecting that path. If something threatens it, that something loses.

Why two laptops: the heatmap updating on a different machine while the student clicks is the moment that proves the system is real and not a slideshow. Rehearse it.

---

3. The team

Six people. One person per area, and - critically - **one person per file**. Merge conflicts are the single most common way a hackathon team loses hours.

#	Role	Owns these files
1	Backend + AI engine (Sushree)	all of backend/, all of prompts/
2	Frontend - Student	frontend/src/pages/student/
3	Frontend - Teacher	frontend/src/pages/teacher/
4	Frontend - Admin + Auth	frontend/src/pages/admin/, frontend/src/pages/auth/
5	Content, language, accessibility	backend/data/ source PDFs, seed content, misconception list, i18n + a11y
6	Integration, demo, deck	docs/demo-script.md, glue, the presentation

A note on load

Role 1 is carrying backend *and* the AI engine - roughly 17 of the 32 features, with five people downstream. That is a real bottleneck, and it is a deliberate choice, not an oversight. Two things keep it from sinking the build:

- **The API contract is written first** (section 7), so nobody waits on working code to start.
- **Roles 5 and 6 absorb work that does not need Python**: seed content, the misconception list, demo data, i18n strings, accessibility, and the deck. If role 1 falls behind, that is the slack to pull from.

What role 5 actually does

This role is easy to underestimate. It produces:

- The 3-4 source PDFs that become the knowledge base.
- The diagnostic questions that produce a **predictable** gap for the demo.
- The list of 8-10 **specific** misconceptions with the wrong answers that signal each one.
- Language strings and the accessibility toggles.

The misconception list is the difference between the demo landing and falling flat. "Struggles with forces" is worthless. "Treats constant velocity as implying a net force" is something a physics teacher recognises instantly.

What role 6 actually does

Not just wiring at the end. Checking in with every pair **throughout**, so that at hour 30 the pieces already fit. Also owns the deck - SIH weighs presentation heavily and teams routinely forget to budget for it.

---

4. Setting up your machine

4.1 What everyone installs first

Git - https://git-scm.com/downloads

On Windows this also installs **Git Bash**, which you will need. Git Bash is a terminal that understands Mac/Linux-style commands. To open it: press the Windows key, type `Git Bash`, press Enter. Do **not** use PowerShell or Command Prompt for the commands in this handbook - several of them will fail there.

On Mac or Linux, just use your normal Terminal.

Check it worked - type this and press Enter:

```
git --version
```

You should see something like `git version 2.43.0`. If you see "command not found", Git did not install correctly.

Node.js version 18 or newer - https://nodejs.org (download the "LTS" version)

Check:

```
node --version
npm --version
```

You need `v18.x.x` or higher. If you have an older version, install the new one over it.

4.2 Getting the code - everyone

Open Git Bash (or Terminal), then:

```
cd ~/Documents
git clone https://github.com/SushreeSudiptaJena/AI-Tutor.git
cd AI-Tutor
```

`cd ~/Documents` means "go to my Documents folder". You can use any folder you like. After the clone you will have a folder called `AI-Tutor`.

Check it worked:

```
ls
```

You should see `README.md`, `backend`, `frontend`, `docs`, `init.sh` and others.

4.3 Frontend developers (roles 2, 3, 4) - about 5 minutes

You do not need Python. You do not need the database. You do not need any API keys. That is deliberate: the repository is public, so the fewer people holding live credentials, the safer.

Step 1 - go into the frontend folder:

```
cd frontend
```

Step 2 - install the packages. This downloads about 86 packages and takes 1-3 minutes:

```
npm install
```

Expected output ends with something like `added 86 packages in 2m`. Warnings are normal. Errors are not.

Step 3 - create your local settings file:

```
echo "VITE_API_BASE=http://localhost:8000" > .env.local
```

This creates `frontend/.env.local`. It tells the app where the backend lives. It is gitignored, so it stays on your machine.

Step 4 - start the app:

```
npm run dev
```

Expected output:

```
VITE v5.4.21  ready in 400 ms
?  Local:   http://localhost:5173/
```

Step 5 - open http://localhost:5173 in your browser.

You will see the baseline screen. It will say the backend is unreachable - **this is correct and expected** when the backend is not running. You build your screens against mock data (section 9.4) and do not need the backend at all for most of your work.

To stop the server, press `Ctrl+C` in the terminal.

4.4 Backend + AI (role 1) - about 10 minutes

Step 1 - from the `AI-Tutor` folder, create your secrets file:

```
cp .env.example .env
```

Step 2 - open `.env` in any text editor and fill in the real values. You need:

- `DATABASE_URL` - the shared Neon Postgres connection string
- `GLM_API_KEY`, `FALLBACK_API_KEY_GEMINI`, `FALLBACK_API_KEY_GROQ`, `SARVAM_API_KEY`

Get these from the team channel. **Never commit this file.** It is gitignored, and the repository is public.

Everything installs from `backend/requirements.txt`, including the embedding model runtime (`fastembed`, which uses ONNX - about 150 MB, and **no PyTorch**). The model files themselves download on first use.

Step 3 - run the setup script:

```
./init.sh
```

This creates a Python virtual environment in `.venv`, installs all Python and Node packages, runs the tests, and prints the commands to start the app. It takes 3-5 minutes the first time.

Expected output ends with:

```
Verification passed.

==> Start command
...
```

If it says `VERIFICATION FAILED`, stop and fix that before doing anything else. A broken baseline outranks every feature.

Step 4 - start the backend:

```
.venv/Scripts/python.exe -m uvicorn app.main:app --reload --port 8000 --app-dir backend
```

On Mac or Linux the path is `.venv/bin/python` instead of `.venv/Scripts/python.exe`.

Expected output:

```
INFO:     Uvicorn running on http://127.0.0.1:8000
INFO:     Application startup complete.
```

Step 5 - check it in another terminal:

```
curl http://localhost:8000/health
```

Expected: `{"status":"ok","db":"ok"}`

If it says `"db"` is something else, your `DATABASE_URL` is wrong or the database is unreachable. Note the app still returns 200 - that is deliberate, so a database problem does not look like a dead app.

4.5 Roles 5 and 6

Role 5 (content) needs only Git and a text editor to start - the source PDFs, the misconception list, and language strings are all plain files. If you later want to run the app, follow 4.3.

Role 6 (integration) should do **both** 4.3 and 4.4, because your job is proving the two halves meet.

4.6 Connecting the frontend to a real backend

For most of the build, frontend developers use mock data and need nothing from anyone.

When you want real data, the backend is running on Sushree's laptop, which your laptop cannot reach directly. She exposes it with a tunnel:

```
cloudflared tunnel --url http://localhost:8000
```

That prints a public address like `https://random-words-here.trycloudflare.com`, which she posts in the team channel.

To use it, edit `frontend/.env.local`:

```
VITE_API_BASE=https://random-words-here.trycloudflare.com
```

Then **stop and restart** `npm run dev` - Vite reads that file at startup.

Important: that tunnel address changes every single time the tunnel restarts. If your app suddenly cannot reach the backend, check the channel for a new address first. Before demo day we deploy the backend to Render's free tier so it has one permanent address that does not depend on anyone's laptop staying awake.

4.7 If something breaks

Symptom	Cause and fix
'.' is not recognized...	You are in PowerShell or CMD. Open Git Bash instead.
./init.sh: Permission denied	Run chmod +x init.sh then try again.
python: command not found	Python is not installed or not on PATH. Reinstall and tick "Add Python to PATH".
npm ERR! code ENOENT	You are in the wrong folder. cd into frontend first.
Port 5173 already in use	Another dev server is running. Close it, or Vite will offer port 5174.
Port 8000 already in use	An old backend is still running. Close that terminal.
Frontend says backend unreachable	Normal if the backend is not running. Otherwise check VITE_API_BASE and restart npm run dev.
.venv is broken	Delete it (rm -rf .venv) and run ./init.sh again. Nothing is lost - it is gitignored.

$'\r': command not found	Line-ending problem. Should not happen - .gitattributes prevents it. Report it.

5. Project structure

```
AI-Tutor/
|
+-- README.md                  what this project is, quick start
+-- CLAUDE.md                  operating rules for anyone (or any AI) working here
+-- claude-progress.md         where the project stands right now - read before you start
+-- feature_list.json          all 32 features, status, and how to verify each
+-- init.sh                    installs everything, runs tests, prints start commands
+-- pytest.ini                 test configuration
+-- .env.example               template for secrets - copy to .env and fill in
+-- .gitignore                 what git must never track
+-- .gitattributes             forces LF line endings so scripts work on every OS
|
+-- docs/
|   +-- handbook.md            this document
|   +-- api-contract.md        every endpoint and every response shape
|   +-- team-roles.md          who owns what, and how it all merges
|   +-- frontend-guide.md      for the three dashboard developers
|   +-- database-guide.md      the data model, spelled out
|   +-- ai-guide.md            retrieval, alignment, refusal, guardrails
|   +-- demo-script.md         the golden path, written out for rehearsal
|   \-- pdf/                   PDF versions of all of the above
|
+-- prompts/                   LLM prompt templates, plain markdown
|   +-- tutor_explain.md
|   +-- gap_diagnose.md
|   +-- misconception_diagnose.md
|   +-- practice_generate.md
|   +-- evidence_check.md
|   \-- reteach_suggest.md
|
+-- backend/
|   +-- requirements.txt       Python packages
|   +-- app/
|   |   +-- main.py            the FastAPI app - routes get registered here
|   |   +-- config.py          reads .env
|   |   +-- db.py              database connection and session
|   |   +-- models.py          every database table, in one file
|   |   +-- schemas.py         request and response shapes
|   |   +-- routers/           one file per dashboard
|   |   |   +-- auth.py
|   |   |   +-- admin.py
|   |   |   +-- student.py
|   |   |   +-- teacher.py
|   |   |   \-- tutor.py
|   |   +-- services/          the thinking parts
|   |   |   +-- ingest.py         PDF -> page-anchored chunks
|   |   |   +-- embed.py          text -> 384-dim vector
|   |   |   +-- retrieve.py       find relevant chunks
|   |   |   +-- evidence.py       alignment score + refusal decision
|   |   |   +-- guardrails.py     graded-work refusal
|   |   |   +-- misconception.py  diagnose the wrong answer
|   |   |   \-- tutor.py          orchestrates the above
|   |   \-- providers/         talking to AI vendors
|   |       +-- base.py           the common interface
|   |       +-- glm.py            primary
|   |       +-- gemini.py         fallback
|   |       +-- groq.py           fallback
```

```
|   |       +-- mock.py           canned responses, works offline
|   |       +-- cache.py          disk cache - demo insurance
|   |       \-- translate_sarvam.py
|   +-- scripts/
|   |   +-- check_db.py        proves the shared database works
|   |   +-- reset_db.py        drops and recreates every table
|   |   +-- ingest_pdfs.py     runs ingestion over backend/data/
|   |   \-- seed.py            loads the demo data
|   +-- data/                  source PDFs
|   \-- tests/                 must pass with no database and no network
|
+-- frontend/
|   +-- package.json
|   +-- .env.example           copy to .env.local
|   \-- src/
|       +-- main.tsx           entry point
|       +-- App.tsx            routes
|       +-- index.css          Tailwind + accessibility classes
|       +-- lib/
|       |   +-- api.ts         EVERY network call goes through here
|       |   \-- mock.ts        fake data with the contract's shape
|       +-- components/        shared components
|       +-- components/ui/     shadcn components
|       \-- pages/
|           +-- auth/
|           +-- student/
|           +-- teacher/
|           \-- admin/
|
+-- scripts/
|   \-- make_docs_pdf.py       renders docs/*.md to docs/pdf/*.pdf
|
\-- evidence/                  proof each feature actually works
```

The three files that run the project

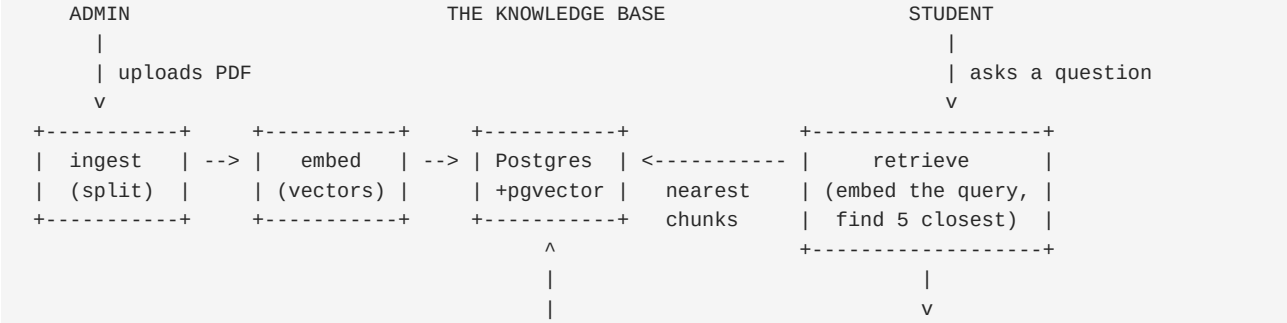
- **`claude-progress.md`** - read this first, every session. It states what is verified working, what is blocked, and what to do next. It is the source of truth, not the code.
- **`feature_list.json`** - all 32 features with `status`, `depends_on`, `verification` steps and `evidence`. Only **one** feature may be `in_progress` at any moment.
- **`CLAUDE.md`** - the operating rules: how to start, how to work, what "done" means.

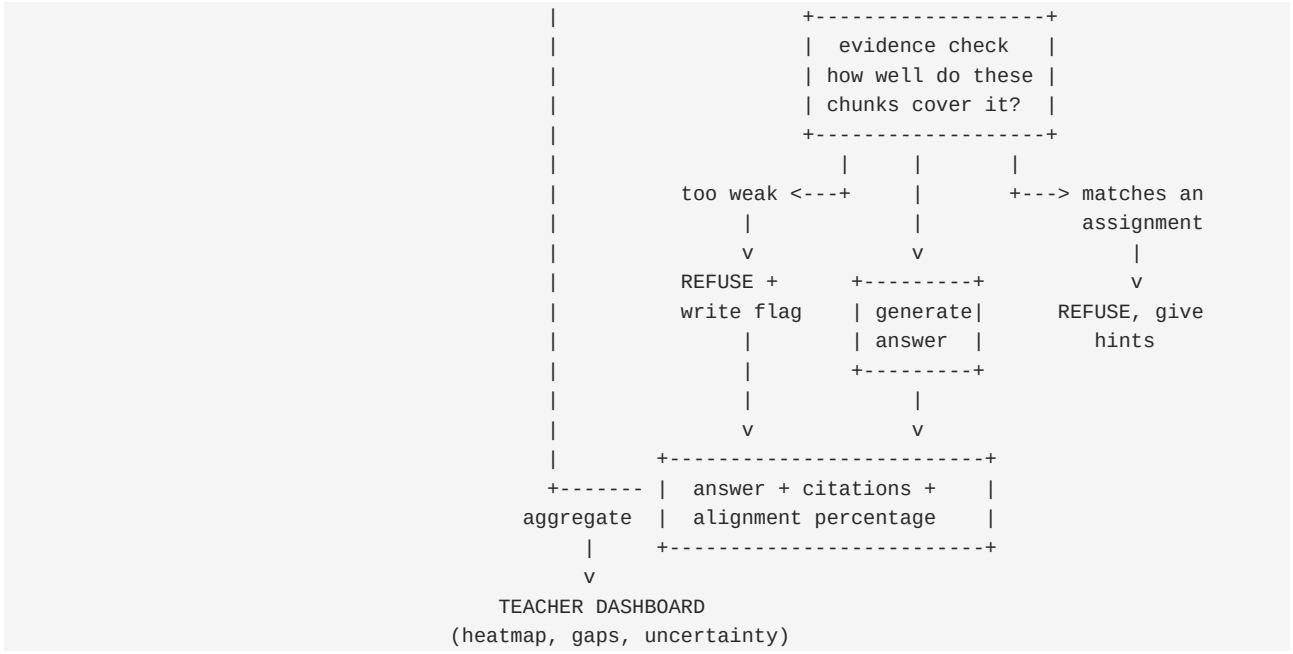
---

6. How the system works - the whole mechanism

This section explains every moving part, what it actually does to the data, and how the parts chain together. Read it once end to end and you will understand the entire project.

6.1 The system in one picture





Everything flows one way: **approved material in, grounded answers out.** The tutor has no other source of knowledge. That constraint is the whole product.

6.2 Ingestion - turning a PDF into searchable pieces

File: `backend/app/services/ingest.py` - **Feature:** `ingest-001`

A 400-page textbook cannot be handed to an AI model in one piece - it is far too large, and even if it fit, the model would have no way to tell you *which page* an answer came from. So we cut it up.

Step 1 - open the PDF and walk it page by page. PyMuPDF gives us text plus the page number it came from:

```
import fitz                      # PyMuPDF

doc = fitz.open(path)
for page_index, page in enumerate(doc):
    text = page.get_text()
    page_no = page_index + 1     # humans count from 1
```

Step 2 - split each page into overlapping chunks. Roughly 800 characters each, overlapping by about 150. The overlap matters: if a sentence explaining a concept straddles a chunk boundary, a non-overlapping split would cut the idea in half and both pieces would retrieve badly.

Step 3 - record where each chunk came from. This is the step everything downstream depends on:

```
Chunk(
    material_id = material.id,
    page_no     = page_no,           # <-- the citation lives or dies here
    chapter     = "5. Newton's Laws of Motion",
    char_start  = 0,
    char_end    = 800,
    text        = "The net force on a body equals the rate of change...",
    embedding   = None,              # filled in by the next stage
)
```

Why the page number is non-negotiable: the problem statement asks the tutor to "cite the source material used". A citation reading "from the textbook" proves nothing to a judge. One reading "Concepts of Physics, Vol 1, page 143" can be opened and checked on the spot. If `page_no` is lost at ingestion, no amount of work later can

recover it.

One special case: material uploaded with `kind="assignment"` is chunked and embedded exactly like everything else, but it is flagged. It becomes searchable for **matching** - that is how the guardrail recognises a homework question - while never being used as a source to quote from.

6.3 Embeddings - turning meaning into numbers

File: `backend/app/services/embed.py`

A computer cannot compare two sentences for *meaning* using string matching. "What is force?" and "Explain Newton's second law" share almost no words but are closely related. Embeddings solve this.

An **embedding model** reads a piece of text and outputs a fixed-length list of numbers - for `bge-small-en-v1.5`, exactly **384 numbers**. Text with similar meaning produces similar numbers. That is the entire trick.

```
"force equals mass times acceleration"  ->  [0.041, -0.118, 0.207, ... ]   384 numbers
"photosynthesis in plant cells"         ->  [-0.233, 0.014, -0.092, ... ]  384 numbers
```

How closeness is measured. Each list of 384 numbers is a point in 384-dimensional space. We measure the *angle* between two such points - **cosine similarity**. Two texts about the same idea point in nearly the same direction.

- Similarity **1.0** = identical meaning
- Similarity **0.8** = closely related
- Similarity **0.1** = unrelated

Postgres gives us `cosine_distance`, which is the opposite: **smaller means more similar**.

```
similarity = 1 - distance
```

This one line is the most common source of a silent bug in this project. Invert it by accident and the tutor confidently retrieves the *least* relevant chunks and reports a high alignment score while doing so. Everything still runs; the numbers are just wrong. Test it against one question you know is covered and one you know is not, and confirm the numbers go the right way.

The similarity floor - measured, not assumed. Two texts about completely unrelated subjects do *not* score near zero. Measured against this model:

Question	Top similarity
covered: "why does a block at constant speed need no net force?"	0.78
covered: "how do I split a vector into components?"	0.73
off-topic, nearby field: "explain Lagrangian mechanics"	0.72
off-topic: "what is photosynthesis?"	0.54
off-topic: "who won the 2018 World Cup?"	0.40

Two consequences, and both are easy to get wrong:

1. **A low refusal threshold never refuses.** Set it at 0.35 and even the World Cup question passes. The `rag-003` feature would silently never fire, and nobody would notice because nothing errors. Our threshold is **0.68**, and `backend/scripts/calibrate_threshold.py` re-derives it against the real corpus once the demo PDFs are ingested.

2. **Similarity alone cannot carry the refusal.** Look at rows 2 and 3: an off-topic question from a nearby field (0.72) outranks a genuinely covered one (0.73) by 0.01. That is why the evidence check has a second signal - the entailment call is load-bearing, not decoration.

The query prefix. BGE models are trained so that *queries* carry a prefix and *documents* do not:

```
QUERY_PREFIX = "Represent this sentence for searching relevant passages: "
embed_query(q)    -> embed(QUERY_PREFIX + q)     # questions
embed_document(d) -> embed(d)                    # chunks, no prefix
```

Measured effect: the gap between the worst covered question and the best off-topic one widens from **+0.012 to +0.050** - four times the separation, for one string concatenation. Applying the prefix to stored documents too would undo the benefit.

Vectors come out unit-normalised (length exactly 1.0), so cosine similarity equals the dot product. Useful to know when reading the numbers.

Every chunk is embedded once, at ingestion. The vectors are stored in the `chunks.embedding` column. A question is embedded once per ask. That asymmetry is why retrieval is fast - the expensive work happened up front.

6.4 Storage - why pgvector instead of a separate vector database

File: `backend/app/db.py`, `backend/app/models.py`

`pgvector` is a Postgres extension adding a `vector` column type and distance operators. Because the embeddings live in the *same* database as courses, users and gaps, a single query can filter by course, exclude assignments, and rank by similarity - no syncing between two systems, no second thing to deploy or debug.

```
embedding: Mapped[list[float]] = mapped_column(Vector(384))
```

Why we deliberately do not build an index. Vector databases usually need an index (`ivfflat`, `hnsw`) to search millions of vectors quickly. Our corpus is a few thousand chunks. Postgres scans all of them in a few milliseconds. An index would add tuning parameters, a build step, and a class of bug where the index silently returns approximate results - for zero benefit at this size.

6.5 Retrieval - finding the right pieces

File: `backend/app/services/retrieve.py` - **Feature:** `rag-001`

```
def search(db, query: str, *, limit=5, kinds=None):
    vec = embed(query)                                  # 1. question -> 384 numbers

    stmt = (select(Chunk, Chunk.embedding.cosine_distance(vec).label("dist"))
            .join(Material)
            .order_by("dist")                           # 2. nearest first
            .limit(limit))                              # 3. top 5

    if kinds:
        stmt = stmt.where(Material.kind.in_(kinds))     # guardrail: assignments only
    else:
        stmt = stmt.where(Material.kind != "assignment") # normal: never quote homework

    return [Hit(chunk=c, similarity=1 - d) for c, d in db.execute(stmt).all()]
```

Four things happen:

1. The question becomes a vector using the **same model** that embedded the chunks. Using a different model

here would be like measuring one thing in metres and the other in feet - the numbers are incomparable.
2. Postgres computes the distance from that vector to every chunk.
3. It sorts and returns the closest five.
4. Assignments are excluded by default, so homework text can never end up quoted as a lesson.

The returned chunks carry their `page_no` and `book_title` with them. **Those become the citations** - we are not asking the AI to remember where it read something, we already know, because we never lost track.

6.6 The evidence check - where the alignment score comes from

File: `backend/app/services/evidence.py` - **Feature:** `rag-002`

This is the feature that makes the product defensible. Retrieval always returns *something* - even for a question the curriculum never covers, five chunks come back, just poorly matched. So we have to judge the quality of the match, not merely its existence.

Two signals, combined:

Signal 1 - retrieval similarity. How close was the best chunk?

```
top = max(hit.similarity for hit in hits) if hits else 0.0
```

This is cheap and catches the obvious case: nothing in the corpus is remotely about this.

Signal 2 - entailment. Do these chunks actually *support* the answer we are about to give? One small LLM call:

> *Here are five excerpts from the approved textbook. Here is a proposed answer. On a scale of 0 to 1, how fully is this answer supported by these excerpts alone? Reply with only a number.*

This catches the subtler failure: chunks that are topically close but do not contain the specific fact - the situation where a model would normally fill the gap with plausible invention.

Combine them:

```
score   = 0.6 * top + 0.4 * entail
percent = round(score * 100)                # what the student sees
```

The weighting is a judgement call, not a law. Retrieval is weighted higher because it is objective; entailment is a model's opinion, useful but softer.

Do not be tempted to drop the entailment call to save time or tokens. Section 6.3 shows why: an off-topic question from a neighbouring field scores within 0.01 of a covered one on retrieval similarity alone. Signal 1 catches the World Cup question. Only signal 2 catches "explain Lagrangian mechanics" - and that second kind is exactly what a judge will try.

The output object drives three different behaviours:

```
EvidenceReport(
    alignment_score   = score,
    alignment_percent = percent,          # -> the badge on the lesson card
    top_similarity    = top,
    threshold         = THRESHOLD,        # 0.35 by default
    sufficient        = score >= THRESHOLD,   # -> answer, or refuse
    reason            = None if sufficient else "no_matching_material",
)
```

One computation feeds the student's badge, the refusal decision, and the teacher's uncertainty flag. That is why it lives in one place.

6.7 The refusal mechanism - knowing when to stop

File: `backend/app/services/tutor.py` - **Feature:** `rag-003`

```
if not report.sufficient:
    flag = UncertaintyFlag(
        question        = question,
        alignment_score = report.alignment_score,
        reason          = report.reason,
        topic_id        = topic_id,
    )
    db.add(flag)
    db.flush()                      # so we have flag.id to return

    return TutorResult(
        outcome             = "insufficient_evidence",
        body                = "I don't have approved course material covering this. "
                              "I've flagged it for your teacher.",
        citations           = [],
        evidence            = report,
        uncertainty_flag_id = flag.id,
    )
```

Three details that matter more than they look:

It returns HTTP 200, not an error. A refusal is a *successful* response - the system did exactly the right thing. Sending a 4xx or 5xx would make correct behaviour look like a crash, and would make frontend error handling swallow it.

The same write populates the teacher dashboard. No separate reporting pipeline, no cron job. The student's refusal *is* the teacher's data. One feature, two dashboards.

No `user_id` is stored on the flag. Teacher views must be anonymous, and the cheapest way to guarantee that is to never record the link. You cannot leak what you never wrote down.

6.8 The graded-work guardrail

File: `backend/app/services/guardrails.py` - **Feature:** `rag-004`

The problem statement requires the tutor to "avoid completing graded work on behalf of students". The mechanism reuses machinery we already have.

```
def is_graded_work(db, question) -> Material | None:
    hits = search(db, question, kinds=["assignment"], limit=1)
    if hits and hits[0].similarity > 0.80:
        return hits[0].chunk.material
    return None
```

Assignment PDFs were embedded alongside everything else, so asking "is this question *in* an assignment?" is just a search restricted to `kind="assignment"`.

The threshold is high - 0.80 - on purpose. We want near-verbatim matches. Set it low and the tutor starts refusing every question about projectile motion merely because projectile motion also appears on a problem set. A student asking "why does a projectile follow a parabola?" deserves a real answer. A student pasting "Q3: A ball is thrown at 20 m/s at 35 degrees; find the range" does not.

When it triggers, the tutor does not go silent - it teaches:

```
return TutorResult(
    outcome            = "graded_work_refused",
    body               = "This looks like it's from a graded assignment, so I won't "
```

```
                          "solve it. Here's how to approach it.",
    hints              = generate_hints(chunks),     # method, not answer
    citations          = chunks_to_citations(hits),
    matched_assignment = {"material_id": m.id, "title": m.title},
)
```

That distinction - refusing the answer while still helping - is what makes it a tutor rather than a filter.

6.9 Generation - writing the explanation

Files: `backend/app/services/tutor.py`, `prompts/tutor_explain.md`

Only once the evidence check passes and the guardrail is clear do we ask a model to write anything.

The prompt is assembled from three parts:

1. **System rules** (from `prompts/tutor_explain.md`) - the constraints.
2. **The retrieved chunks** - labelled with their book and page.
3. **The student's question.**

```
Use ONLY the provided source material. Do not use outside knowledge.
If the sources do not contain the answer, say so - do not guess.
Cite the page number for each claim.
Never provide a final answer to a graded assignment question.
Return JSON matching the given schema. No prose outside the JSON.

SOURCES:
[1] Concepts of Physics, Vol 1, p.143: "The net force on a body equals..."
[2] Concepts of Physics, Vol 1, p.144: "When a body moves with constant velocity..."

QUESTION: Why doesn't a block sliding at constant speed need a net force?
```

Grounding is enforced twice, deliberately. The prompt *asks* the model to use only the sources; the evidence check then *verifies* the result actually follows from them. Prompts alone are a request, not a guarantee - models drift. The verification step is what turns a polite instruction into a property of the system.

Prompts live as plain `.md` files so they can be edited and iterated without touching Python.

6.10 Citations - how the page number survives

There is no clever step here, and that is the point. The page number is carried, never re-derived:

```
PDF page 143
   -> ingest records page_no=143 on the chunk
   -> the chunk is embedded and stored, page_no intact
   -> retrieval returns that chunk row, page_no intact
   -> the API serialises it into the citations array
   -> the UI prints "Concepts of Physics, Vol 1, p.143"
```

At no point does anyone ask a language model where something came from. **The AI never generates a citation.** It writes prose; the citation is a database fact attached alongside. That is why our citations cannot hallucinate - structurally, they are incapable of it.

This is worth saying explicitly in the pitch, because "the AI cites its sources" usually means "the AI produces text that looks like a citation", which is a very different and much weaker claim.

6.11 Gap detection - finding what a student is missing

Files: `backend/app/routers/student.py` - **Feature:** `student-002`

Two entry points, one output.

Path A - the diagnostic. Each `DiagnosticItem` is tied to a `concept`, and each concept can be tied to a `prerequisite_course`. The student answers; wrong answers point at concepts:

```
for answer in submitted:
    item = get_item(answer.item_id)
    if answer.value != item.correct_answer:
        db.add(Gap(user_id=user.id, concept_id=item.concept_id,
                   detected_from="diagnostic", status="open"))
```

Path B - syllabus upload. An incoming student uploads their previous syllabus. We extract its topics and compare against the concepts this course assumes. Whatever the course expects but the syllabus never covered becomes a gap.

The output is a list of named concepts, and nothing else. No score, no percentage, no pass/fail. This is a deliberate product decision taken straight from the problem statement - "identifies prerequisite gaps", not "grades students". There is no score column in the database to store one even if someone wanted to.

Why it matters beyond compliance: a student who sees "you scored 4/10" feels judged and stops. A student who sees "you're missing: vector components, free-body diagrams" has a to-do list.

6.12 Scoped practice generation

File: `backend/app/services/tutor.py` - **Feature:** `student-005`

The point is **scoped** - problems for *this student's* specific gap, not random questions from the whole course.

```
gap ("Vector components")
   -> retrieve chunks about that concept from approved material
   -> prompt: "write 3 practice questions using ONLY these excerpts,
               tag each with problem_type, include the correct answer"
   -> store as PracticeItem rows, each carrying gap_id and problem_type
```

`problem_type` is the field that makes misconception diagnosis possible later - it is how we know which known error patterns could apply to this question.

`correct_answer` is stored but never sent to the client. It is stripped in the response model. Otherwise a student could read the answers out of the network tab, which would rather undercut the whole exercise.

6.13 Misconception diagnosis - the heart of the demo

File: `backend/app/services/misconception.py` - **Feature:** `student-006`

A normal quiz says "wrong, the answer is 12 N". This system says *why you got it wrong* - it names the incorrect mental model behind that specific answer.

Stage 1 - pattern match. Seeded misconceptions carry a `wrong_answer_pattern`. If the student's answer matches a known pattern for that `problem_type`, we have the diagnosis with **no AI call at all**:

```
for m in known_for(db, attempt.item.problem_type):
    if m.wrong_answer_pattern and matches(attempt.answer, m.wrong_answer_pattern):
        return record(db, attempt, m)
```

Fast, free, and completely deterministic - which is exactly what you want in a live demo.

Stage 2 - LLM fallback. No pattern matched, so we ask a model to pick from the **known list** for that problem

type. Crucially we give it the candidates rather than letting it invent one; an invented misconception would be unverifiable and would pollute the teacher's heatmap with noise.

Stage 3 - ask the student.

> *"It looks like you assumed constant velocity means there's a net force. Does that match your thinking?" [Yes] [No]*

Only `confirmed = true` reaches the teacher's heatmap. Denied diagnoses are stored but excluded from every aggregate. That is what makes the teacher's number trustworthy: it counts students who *agreed* this was their reasoning, not an algorithm's guesses.

Why the misconception must be specific. "Struggles with forces" tells a teacher nothing. "Treats constant velocity as implying a net force" is a diagnosis a physics teacher recognises instantly and can teach against tomorrow morning. The seeded list is where this demo is won or lost - budget real time for writing 8-10 good ones.

6.14 Aggregation - how the teacher dashboard fills itself

File: `backend/app/routers/teacher.py` - **Features:** `teacher-001` to `teacher-005`

The teacher dashboard runs no AI. It is arithmetic over rows the student flow already wrote.

Teacher view	Where its data comes from
Misconception heatmap	count misconception_diagnoses where confirmed = true, grouped by misconception
Reasoning-path breakdown	the same rows, plus one anonymised example answer
Prerequisite gap map	count gaps grouped by concept, divided by class size
Uncertainty flags	the uncertainty_flags rows written by refusals
Before/after	the same confirmed counts, split into two time windows around a reteach

---

This is why the demo works. When the student confirms a misconception on laptop A, a row is written. The teacher's heatmap on laptop B polls every five seconds, re-runs its count, and the number goes up. **There is no special "demo mode" wiring** - it is the ordinary data path, which is why it will not break under pressure.

Anonymity is structural. Every teacher query aggregates. No teacher endpoint returns a `user_id`, a name, or an email, and `uncertainty_flags` has no user column to leak in the first place. Check this in the JSON payload, not just the rendered page.

Polling, not websockets. Five seconds is imperceptible in a demo and costs us nothing. Websockets would add a connection lifecycle, a reconnect path, and a new failure mode - for no visible gain.

6.15 The reteach loop - closing the circle

Feature: `teacher-006`

```
heatmap shows the top misconception
   -> teacher clicks "suggest reteach"
   -> AI drafts a short lesson + practice, grounded in approved material
   -> status = "draft"          <-- invisible to every student
   -> teacher edits the text
   -> teacher approves
   -> status = "assigned"       <-- now it appears in /student/assignments
   -> students work through it
   -> new confirmed diagnoses accumulate
   -> before/after shows whether the misconception is actually shrinking
```

A `draft` unit must never appear in any student query. The approval gate is the human-in-the-loop story the problem statement asks for - the AI proposes, a teacher decides. Never auto-assign, even if it would demo faster.

The before/after view is the payoff: it measures whether a *specific* misconception is shrinking, rather than whether overall scores went up. Overall scores rise for many reasons, most of them uninformative.

6.16 Multilingual - one vector space, many languages

File: `backend/app/providers/translate_sarvam.py` - **Feature:** `i18n-001`

```
Hindi question
   -> Sarvam translates to English
   -> embed the English, retrieve from the English corpus
   -> evidence check on English
   -> generate the English answer
   -> Sarvam translates the answer back to Hindi
   -> citations stay pointing at the English book and page
```

Why translate rather than use a multilingual embedding model. If the corpus is English and the question is Hindi, you would need a model whose vector space aligns both languages - that is `bge-m3`, roughly 2.2 GB plus PyTorch. Translating instead lets us use a 130 MB English model and keeps exactly one vector space to reason about and debug.

A property worth pointing out in the pitch: because retrieval and the evidence check both run on English, **the alignment score is identical whether the student asked in English or Hindi.** A student working in their own language gets provably the same quality of grounding, not a degraded version. That is a real engineering answer to a real equity problem, and it is the kind of detail that separates a considered system from a translated wrapper.

6.17 The provider layer - why the demo cannot be killed by wifi

Files: `backend/app/providers/` - **Feature:** `infra-004`

Every AI call and every translation goes through one interface. No service ever calls GLM or Sarvam directly.

```
class Provider(Protocol):
    def complete(self, prompt: str, *, system: str = "",
                 json_schema: dict | None = None) -> str: ...
```

Four implementations: `glm.py` (primary), `gemini.py`, `groq.py` (fallbacks), `mock.py` (canned responses).

The cache is the single most valuable component in this repository.

```
key  = sha256(f"{model}\n{system}\n{prompt}".encode()).hexdigest()
path = LLM_CACHE_DIR / f"{key}.json"
if path.exists():
    return json.loads(path.read_text())["text"]
```

Identical input produces a byte-identical answer, served from disk in microseconds. Three consequences:

1. **The demo is repeatable.** The same click produces the same output every rehearsal, so there are no surprises on stage.
2. **The demo is fast.** No waiting for a model mid-presentation.
3. **The demo survives no internet.** Rehearsing warms the cache. With a warm cache and `PROVIDER=mock`, the entire golden path runs with the wifi switched off.

The fallback chain is GLM -> Gemini -> Groq -> mock, deliberately across *different vendors*. A rate limit or outage at one provider cannot take out its own backup.

Test this before demo day: put a garbage value in `GLM_API_KEY` and confirm the answer still arrives.

6.18 Authentication - what happens on every request

Files: `backend/app/routers/auth.py`, `backend/app/models.py` - **Feature:** `auth-001`

Signing up:

```
password -> pbkdf2_sha256 hash -> stored in users.password_hash
```

The plaintext password is never stored, and never logged. We use `pbkdf2_sha256` from Python's standard library rather than bcrypt because it needs no compiler - bcrypt regularly fails to build on Windows, and losing an hour to that during a 36-hour build is a bad trade for no security benefit at this scale.

Logging in:

```
verify hash -> create Session(token=uuid4().hex, user_id=...) -> return the token
```

Every subsequent request:

```
Authorization: Bearer <token>
   -> look the token up in the sessions table
   -> no row?   -> 401
   -> row found -> load the user, check their role matches the route
   -> proceed
```

Why an opaque token instead of a JWT. A JWT carries signed claims and cannot be revoked without extra machinery, so logout becomes awkward. Our token is a random string that means nothing by itself - logging out deletes the row and it is instantly dead. Simpler to build, simpler to reason about, and logout genuinely works.

Frontend must not attempt to decode it. To learn who is logged in, call `/auth/me`.

6.19 One request, traced all the way through

A student clicks a gap. Here is every step.

```
1. BROWSER    GET /student/gaps/21/lesson
              Authorization: Bearer 8f2a...

2. CORS       middleware confirms the origin is localhost or *.trycloudflare.com

3. AUTH       token -> sessions table -> user id 7, role "student"
              wrong role or no row -> 401, stop here

4. ROUTER     student.py loads gap 21, confirms it belongs to user 7
              reads the concept: "Vector components"

5. LANGUAGE   user.preferred_language == "hi"
              -> Sarvam translates the concept query to English

6. GUARDRAIL  search restricted to kind="assignment"
              best similarity 0.31 -> below 0.80 -> not homework, continue

7. RETRIEVE   embed the query -> 384 numbers
              Postgres ranks every chunk by cosine distance
              returns 5 nearest, excluding assignments
              best similarity 0.79

8. EVIDENCE   top = 0.79
              entailment call -> 0.86        (cache hit: served from disk)
```

```
               score = 0.6*0.79 + 0.4*0.86 = 0.818
               0.818 >= 0.35 -> sufficient

9. GENERATE    prompt = rules + 5 labelled chunks + the question
               provider: GLM (cache hit -> instant, identical to last time)
               returns grounded prose

10. TRANSLATE answer back to Hindi. Citations untouched - they stay English.

11. SERIALISE {
                 "outcome": "answered",
                 "language": "hi",
                 "body": "<Hindi text>",
                 "citations": [
                   {"book_title": "Concepts of Physics, Vol 1", "page_no": 143, ...}
                 ],
                 "evidence": {"alignment_percent": 82, "sufficient": true}
               }

12. BROWSER    api.ts parses it
               switch on outcome -> "answered"
               renders the lesson, the "82% syllabus aligned" badge,
               and "Show Source: Concepts of Physics, Vol 1, p.143"
```

Note what happened at steps 6 and 8: **the tutor had to earn the right to answer.** It proved the question was not homework, and proved the material actually covers it, before a single word was generated.

6.20 The golden path, through every component

The demo, mapped onto the machinery above.

#	What the audience sees	What actually runs
1	Student logs in, sees course scope	auth-001 -> student-001: sessions table, then materials and topics for their course
2	Takes the diagnostic, gets a gap list	student-002: wrong answers -> concepts -> gaps rows. No score is computed.
3	Opens a gap, sees a lesson at 82%	rag-001 retrieval -> rag-002 evidence -> generation -> citations from page_no
4	Clicks Show Source, sees book + page	student-004: the page number carried since ingestion, never re-derived
5	Asks something off-syllabus, is refused	rag-003: score below threshold -> refusal + uncertainty_flags row
6	(laptop B) the flag is already there	teacher-004: reading the row step 5 wrote. No extra wiring.
7	Pastes an assignment question, gets hints	rag-004: assignment similarity above 0.80 -> hints instead of an answer
8	Practises, answers wrong	student-005: practice scoped to the gap, tagged problem_type
9	AI names the exact misconception	student-006: seeded pattern match, no AI call needed
10	Student confirms it	confirmed = true written
11	(laptop B) heatmap increments live	teacher-001: poll re-runs the count. Ordinary data path.

12	Teacher approves a reteach unit	teacher-006: draft -> assigned, only then student-visible
13	Switch to Hindi, same score	i18n-001: translate in/out, English vector space, score unchanged

Thirteen visible moments. Every one of them is a normal code path - nothing is special-cased for the demo, which is exactly why it will hold up when a judge asks you to do it again with a different question.

7. The API contract

The full version is `docs/api-contract.md` - 46 endpoints. This section covers what everyone needs to know.

The contract is frozen at hour 4. After that, you change this document first, announce it in the channel, then change code. That rule is what lets six people work at once without waiting for each other.

Conventions

Base URL	whatever VITE_API_BASE is set to
Format	JSON, snake_case field names
Auth	header Authorization: Bearer <token>
Timestamps	ISO 8601 UTC, e.g. 2026-08-23T11:48:25Z
Lists	{ "items": [...], "total": 12 }

Errors

Every failure looks the same:

```
{ "error": { "code": "not_found", "message": "No such route.", "detail": {} } }
```

Status	Meaning
400	malformed request
401	not logged in, or token expired
403	logged in but wrong role
404	does not exist
409	conflict, e.g. email already registered
422	validation failed, detail says which field
503	every AI provider failed, including fallbacks

The objects everyone uses

User

```
{ "id": 1, "email": "asha@example.edu", "full_name": "Asha R",
  "role": "student", "course_id": 3, "preferred_language": "en" }
```

Citation - attached to anything the tutor produces. Never empty on a real answer.

```
{ "chunk_id": 812, "material_id": 4,
  "book_title": "Concepts of Physics, Vol 1",
  "page_no": 143, "chapter": "5. Newton's Laws of Motion",
  "snippet": "the net force on a body equals..." }
```

EvidenceReport - the alignment score and the refusal decision in one object.

```
{ "alignment_score": 0.82, "alignment_percent": 82,
  "top_similarity": 0.79, "threshold": 0.35,
  "sufficient": true, "reason": null }
```

`alignment_percent` is what the UI shows.

TutorResponse - read this part twice.

Every tutor reply has an `outcome` field with **three** possible values, and **all three arrive as HTTP 200**:

```
{ "outcome": "answered", "language": "en",
  "body": "A vector's components are...",
  "citations": [ ... ], "evidence": { ... } }
```

```
{ "outcome": "insufficient_evidence", "language": "en",
  "body": "I don't have approved course material covering this. I've flagged it for your teacher
.",
  "citations": [], "evidence": { "sufficient": false, "alignment_percent": 11 },
  "uncertainty_flag_id": 55 }
```

```
{ "outcome": "graded_work_refused", "language": "en",
  "body": "This looks like it's from a graded assignment, so I won't solve it.",
  "hints": ["Start by resolving the force into components.", "..."],
  "citations": [ ... ],
  "matched_assignment": { "material_id": 9, "title": "Problem Set 3" } }
```

If a screen only handles `answered`, the two features the judges care most about render as blank cards. The build will look broken at exactly the moment it is working correctly. Handle all three.

Endpoints by area

System

Method	Path	Returns
GET	/health	{"status":"ok","db":"ok"} - always 200 while the app is up
GET	/meta/provider-status	which AI provider is active, whether the cache is on
GET	/languages	the language list

Auth

Method	Path	Body
POST	/auth/signup	{ email, password, full_name, role, course_id? }
POST	/auth/login	{ email, password } -> { token, user }
POST	/auth/logout	-
GET	/auth/me	-> User
PATCH	/auth/me/preferences	{ preferred_language }

The token is an opaque string. **It is not a JWT - do not try to decode it.** To find out who is logged in, call `/auth/me`.

Forgot password has **no endpoint**. It is a screen that collects an email and shows a confirmation message, and makes no network call at all. Its absence is intentional.

Admin

Method	Path	Purpose

GET/POST	/admin/departments	departments
GET/POST	/admin/courses	courses and their prerequisites
POST	/admin/courses/{id}/materials	upload a PDF (multipart)
GET	/admin/courses/{id}/materials	list, ?include_archived=
POST	/admin/materials/{id}/archive	archive, never delete
GET	/admin/materials/{id}/versions	version history
POST	/admin/materials/{id}/ingest	start chunking + embedding
GET	/admin/ingest-jobs/{id}	poll progress
GET	/admin/audit-log	who did what, when

Student

Method	Path	Purpose
GET	/student/course-summary	what is in scope: books, pages, topics
GET	/student/diagnostic	the prerequisite test
POST	/student/diagnostic/{id}/submit	-> a gap list, not a grade
POST	/student/syllabus-upload	alternative entry for incoming students
GET	/student/gaps	current gaps
GET	/student/gaps/{id}/lesson	-> TutorResponse with alignment + citations
GET	/student/mastery	per-concept solid/shaky
POST	/student/practice/generate	practice for one gap
POST	/student/practice/{id}/answer	-> correctness + misconception diagnosis
POST	/student/misconception-diagnosis/{id}/confirm	{ confirmed: true }
POST	/tutor/ask	-> TutorResponse
GET	/student/assignments	approved reteach units

The diagnostic result contains **no score and no percentage**. The problem statement asks for a gap list, not a grade. Do not compute one in the UI either.

Teacher - every response is anonymised. No student id, name, or email in any field.

Method	Path	Purpose
GET	/teacher/misconceptions/heatmap	ranked by confirmed count
GET	/teacher/problems/{type}/reasoning-paths	top error patterns + one example each
GET	/teacher/gap-map	class-level missing prerequisites
GET	/teacher/uncertainty-flags	where the tutor refused
POST	/teacher/uncertainty-flags/{id}/resolve	mark handled
GET	/teacher/misconceptions/{id}/before-after	is it shrinking after reteach
POST	/teacher/reteach/suggest	AI drafts a reteach unit
PATCH	/teacher/reteach/{id}	teacher edits it
POST	/teacher/reteach/{id}/approve	only then can a student see it
GET	/teacher/verification-queue	AI-found web content awaiting approval
POST	/teacher/verification-queue/{id}/approve	or /reject

8. The database

Shared Postgres with pgvector - **PostgreSQL 18.6, pgvector 0.8.6**, confirmed working. One instance for the whole team, so whatever one person ingests, everyone sees.

Full model definitions are in `docs/database-guide.md`. The main tables:

Table	Holds
departments, courses, course_prerequisites	institution structure
users, sessions	login
materials	uploaded PDFs, with version and status
chunks	text pieces + page_no + a 384-number embedding
topics, concepts	the curriculum map
diagnostic_items	the prerequisite test
gaps, mastery	what a student is missing / has learned
practice_sets, practice_items, attempts	practice and answers
misconceptions, misconception_diagnoses	the heart of the demo
uncertainty_flags	where the tutor refused
reteach_units, sourced_content, audit_log	teacher and admin views

Four deliberate design decisions

1. **No score column, anywhere.** Not on the diagnostic, not on mastery. The problem statement asks for a gap list, not a grade.
2. **No time-on-task column, anywhere.** Teacher dashboards are about misconceptions, not surveillance.
3. **`uncertainty_flags` has no `user_id`.** The cheapest way to guarantee teacher views are anonymous is to never store the link in the first place.
4. **`misconception_diagnoses.confirmed` is three-state** - `null` (asked, not answered), `true`, `false`. **Only `true` feeds the teacher heatmap.** Denied diagnoses are kept but excluded from every aggregate. That is what makes the number honest.

Someone will eventually try to "helpfully" add a score column or a last-seen timestamp. Do not let them.

Rules

- **No Alembic.** Schema changes = edit `models.py`, run `reset_db.py`.
- **`reset_db.py` drops everything, for everyone.** Only Sushree runs it, and announces it first.
- **No vector index.** The corpus is small; brute-force cosine is instant, and an index is one more thing that can go wrong.
- `cosine_distance` returns **distance**. Similarity is `1 - distance`. Getting this backwards silently inverts the alignment score - the demo still runs, the numbers are just wrong.

9. Frontend

9.1 The stack

- **Vite + React + TypeScript**, with `strict` deliberately **off**. Types are documentation here, not a proof system. If TypeScript fights you, write `any` and move on.

- **Tailwind v4** - utility classes in `className`. No config file needed.
- **shadcn/ui** - copy-paste components. Add them with:

```
cd frontend
npx shadcn@latest add button card badge table tabs dialog input
```

9.2 Every network call goes through lib/api.ts

Never write a bare `fetch` in a component.

```
import { api, User, ApiError } from "@/lib/api";

const user = await api<User>("/auth/me");

const { token } = await api("/auth/login", {
  method: "POST",
  body: { email, password },
  auth: false,
});
```

`api()` handles the token, JSON encoding, and turns errors into a thrown `ApiError` with `.status`, `.code`, `.message`.

9.3 Handling the three outcomes

```
switch (res.outcome) {
  case "answered":
    return <Lesson body={res.body} citations={res.citations}
                   percent={res.evidence.alignment_percent} />;
  case "insufficient_evidence":
    return <NoEvidenceCard message={res.body} />;
  case "graded_work_refused":
    return <GuardrailCard message={res.body} hints={res.hints} />;
}
```

TypeScript helps: `res.hints` only exists on the third branch, so the compiler tells you if you forgot to narrow.

9.4 Working before the backend exists

Do not wait. Build against a fake with the contract's shape.

```
// src/lib/mock.ts
export const MOCK_LESSON = {
  outcome: "answered",
  language: "en",
  body: "A vector can be split into perpendicular components...",
  citations: [{ chunk_id: 1, material_id: 4,
    book_title: "Concepts of Physics, Vol 1", page_no: 143,
    chapter: "5. Newton's Laws", snippet: "..." }],
  evidence: { alignment_percent: 82, sufficient: true, reason: null },
};
```

```
const USE_MOCK = true;
const lesson = USE_MOCK ? MOCK_LESSON : await api(`/student/gaps/${id}/lesson`);
```

When the real endpoint lands, flip the flag. Nothing else changes, because the shape never changed. **Mock all three outcomes**, not just the happy one.

9.5 Design tokens - so three dashboards look like one app

Purpose	Class
Page wrapper	mx-auto max-w-5xl p-6
Page title	text-2xl font-semibold
Section title	text-lg font-medium mt-8 mb-3
Muted text	text-sm opacity-70
Card	rounded-lg border p-4
Card grid	grid gap-4 md:grid-cols-2
Warning / refusal	border-amber-400 bg-amber-50
High alignment	text-emerald-700
Low alignment	text-amber-700

Use the same alignment badge everywhere:

```
function AlignmentBadge({ percent }: { percent: number }) {
  const tone = percent >= 70 ? "text-emerald-700" : "text-amber-700";
  return <span className={`text-xs font-medium ${tone}`}>{percent}% syllabus aligned</span>;
}
```

9.6 Show Source

Every lesson card needs this. Cheap, and quoted directly in the problem statement.

```
{citations.map((c) => (
  <div key={c.chunk_id} className="text-xs opacity-70">
    {c.book_title}, p.{c.page_no}{c.chapter && ` - ${c.chapter}`}
  </div>
))}
```

9.7 Accessibility - about an hour, worth a whole judging criterion

```
// read aloud, free, no API
function speak(text: string) {
  window.speechSynthesis.cancel();
  window.speechSynthesis.speak(new SpeechSynthesisUtterance(text));
}

// font size and contrast - the CSS classes already exist
document.documentElement.classList.toggle("text-lg-mode");
document.documentElement.classList.toggle("high-contrast");
```

Also: every control reachable by Tab, `alt` on every image, and never signal meaning with colour alone - put the number next to the colour.

9.8 Live updates

Poll. There are **no websockets** in this build.

```
useEffect(() => {
  const load = () => api("/teacher/misconceptions/heatmap?...").then(setData);
  load();
  const t = setInterval(load, 5000);
  return () => clearInterval(t);
}, []);
```

9.9 Before you push

```
npm run build
```

If that fails, `main` breaks for everyone.

10. The AI engine

The three features that win the rubric

Build these by hour 8. If time runs out, they are the **last** things to cut.

1. Alignment score - retrieval similarity plus one entailment check:

```
score = 0.6 * top_similarity + 0.4 * llm_entailment(chunks, answer)
percent = round(score * 100)
```

2. Refuse without evidence - and write the flag:

```
if score < THRESHOLD:
    db.add(UncertaintyFlag(question=q, alignment_score=score,
                           reason="no_matching_material"))
    return TutorResult(outcome="insufficient_evidence", ...)
```

That one write also fills the teacher's Uncertainty Flags panel. One feature, two dashboards.

3. Graded-work guardrail - if the question near-matches a chunk from a material with `kind="assignment"` (similarity above ~0.80), refuse and return hints. Keep the threshold high: a conceptual question about a topic that also appears in an assignment must still get a real answer.

Retrieval

Brute-force cosine over pgvector, no index. Assignments are **matchable but never quotable** - exclude them from normal retrieval.

The provider layer

Every AI and translation call goes through `backend/app/providers/`. Never call GLM or Sarvam directly from a service.

Cache everything, keyed on `sha256(model + system + prompt)`. This is the single most valuable thing in the project:

- The demo replays instantly and identically.
- With a warm cache and `PROVIDER=mock`, the whole golden path runs **with the wifi off**.
- Every rehearsal makes the real demo faster and safer.

Fallback order: GLM -> Gemini -> Groq -> mock. Different vendors on purpose, so one outage cannot take out two.

Misconception diagnosis - the demo's closing beat

Two stages, cheapest first:

1. **Pattern match.** Seeded misconceptions carry a `wrong_answer_pattern`. If the student's answer matches for that `problem_type`, you have the diagnosis with no AI call - fast and reliable.
2. **LLM fallback.** No pattern matched: give the model the known misconceptions for that problem type and have it pick one. Do not let it invent new ones mid-demo.

Then show confirm/deny. Only confirmed diagnoses reach the heatmap.

Multilingual

Translate in -> retrieve in English -> answer in English -> translate out. The vector space stays English-only,

which is why multilingual embeddings are unnecessary.

Citations always point at the English source book and page, and the alignment score is computed on the English text - so **the score does not drift between languages**. Say that out loud in the pitch; it is a real engineering answer to a real problem.

---

11. How we work together

Branches

One branch per feature, named after its id:

```
git checkout -b feat/student-003-gap-lesson
```

Work, commit small and often, push at least every hour so nothing is trapped on one laptop:

```
git add .
git commit -m "student-003: gap list renders with alignment badge"
git push -u origin feat/student-003-gap-lesson
```

Then open a pull request on GitHub. `main` must always be green.

Before you push, run the check for your area - `npm run build` for frontend, `./init.sh` for backend.

Never commit

`.env` - `.env.local` - `node_modules/` - `.venv/` - anything with a password or key in it.

The repository is public. A leaked key is live within minutes of being pushed. If it ever happens, the only real fix is rotating the key - tell Sushree immediately, do not try to quietly delete the commit.

Merge order

1. **Skeleton and contracts** - repo, design tokens, models, API contract, merged first, together. *(Done.)*
2. **Parallel build** - everyone on their own branch, in their own folder, merging small and often.
3. **Real-logic swap** - backend replaces canned responses with real ones. The shape never changed, so frontend touches nothing.
4. **Integration and demo** - golden path fully live, everything else seeded, rehearsed twice.

What "done" means

A feature is `passing` in `feature_list.json` only when **all** of these are true:

1. The `verification` steps in its entry were run exactly as written.
2. **Evidence is saved** in `evidence/<feature-id>/` - the actual terminal output or a screenshot, not a description of it.
3. `./init.sh` still passes.
4. It is committed.

If you cannot produce evidence, it is not passing. Only **one** feature may be `in_progress` at a time.

Seed over build

Only the golden path must be fully live. Everything marked `"demo_mode": "seeded"` in `feature_list.json` ships as realistic pre-loaded data. Building every path to be fully dynamic is how teams run out of time with nothing that demos.

12. Demo day

The plan

Two laptops. Laptop A runs the student flow, laptop B shows the teacher dashboard on a projector.

1. Log in as a student. Show the course scope - real books, real page ranges.
2. Take the prerequisite diagnostic. Get a **gap list, not a grade**.
3. Open a gap. Lesson appears with an **alignment percentage** and **Show Source** pointing at a real page.
4. Ask something off-syllabus. The tutor **refuses** rather than inventing. Switch to laptop B - the flag is already there.
5. Ask it to solve an assignment question. It **declines and gives hints**.
6. Do the practice. Answer wrong. The AI names the **specific misconception**. Confirm it.
7. Switch to laptop B. The **heatmap has updated live**.
8. Teacher generates a reteach unit, edits it, approves it. It appears for the student only after approval.
9. Switch language to Hindi. Ask again. Same citations, same alignment score.

Rules for the last three hours

- **Stop building features.** Rehearse.
- Run the full script twice from a clean state.
- Run it once with the wifi **off** and `PROVIDER=mock`, to prove it survives venue network failure.
- Every rehearsal warms the LLM cache, making the real run faster.
- Have the backend on Render, not on a laptop that might sleep.

Things that have killed hackathon demos before

Risk	Our mitigation
Venue wifi dies	LLM disk cache + PROVIDER=mock
API key hits a rate limit	three vendors: GLM -> Gemini -> Groq
Laptop sleeps mid-demo	backend on Render, not a laptop
Tunnel URL changed	Render gives a permanent address
Empty dashboards look broken	seed.py pre-loads realistic data
Someone's laptop won't run it	it runs on any of the six machines

13. Where the project stands

Feature status lives in `feature_list.json`. As of the last update:

Status	Count
passing	1
blocked	2
not started	29

- **`infra-002` (green baseline) - passing.** `./init.sh` verified on a clean clone; the app boots and `/health` returns 200 with a live database.
- **`infra-001` (shared database) - blocked.** Connectivity confirmed from one machine. Needs a second machine to run `check_db.py --write` to prove it is genuinely shared.
- **`infra-003` (API contract) - blocked.** The document is complete; it needs one frontend developer to build a page against the mock without asking a clarifying question.

Next up is **`infra-004`** - the provider interface with cache, fallbacks and mock. It is the demo's insurance policy and nothing else depends on the blocked items.

---

14. Glossary

Term	Meaning
Alignment score	how directly an answer maps to approved syllabus material, shown as a percentage
Chunk	a piece of a source PDF, tagged with the page it came from
Embedding	a list of 384 numbers representing a piece of text's meaning, so similar text can be found by maths
pgvector	the Postgres extension that stores embeddings and finds nearest matches
RAG	retrieval-augmented generation - find relevant source text first, then have the AI write using only that
Golden path	the one demo flow that must work live end to end
Seeded	pre-loaded realistic data instead of a fully live pipeline
Misconception	a specific wrong mental model, e.g. "treats constant velocity as implying a net force"
Uncertainty flag	a record that the tutor refused for lack of evidence, surfaced to the teacher
Guardrail	the rule that stops the tutor solving graded assignment questions
Tunnel	a temporary public web address for a server running on someone's laptop
Opaque token	a random login string with nothing readable inside it - unlike a JWT
Idempotent	safe to run more than once without changing the result